

Flock Simulation - Project Documentation

Student Name: Nguyen Vo Trong Nguyen

Student Number: 103763778

Degree Program: Data Science

Year of Studies: 1st year

Date: April 21, 2026

1. General Description

This project implements a real-time boid flocking simulation based on Craig Reynolds' foundational model [Reynolds 1987]. The application simulates the emergent collective movement of bird-like agents (boids) governed by three simple local rules: separation, alignment, and cohesion [Reynolds 1999].

Users can observe and interact with a real-time flock rendered in a ScalaFX window, adjusting behavioral weights, saving/loading flock states, and controlling playback.

Compared with the original technical plan, the main changes are:

- (1) the file format was changed from a plain-text CSV to JSON using the Circe library, enabling robust serialization and better compatibility with structured data tooling.
- (2) a dedicated Simulation thread class was not implemented. Instead, the simulation loop is driven directly by a JavaFX AnimationTimer on the FX thread, which proved simpler and avoided synchronization complexity.
- (3) steering behaviors were extracted into a dedicated SteeringBehavior trait hierarchy rather than being methods on Flock, improving extensibility.
- (4) Constants was implemented as a case class rather than a mutable map, ensuring thread-safe parameter sharing.

Given the scope of the project - incorporating a polished GUI, robust file I/O, and comprehensive unit testing - I consider this work to be at the Advanced level.

2. User Interface

Launching the Application

Run the application with: `sbt run` from the project root. The window opens with a default flock loaded from `data/testui.json`, displayed in the simulation canvas.

Window Layout

The main window is divided into three regions:

- **Top:** a menu bar with File and View menus.
- **Center:** the simulation canvas where boids are rendered as small arrow-shaped polygons oriented in the direction of travel.
- **Bottom:** a control panel with parameter sliders, a flock-size spinner, and action buttons.

Control Panel

- Flock size (spinner): adjusts the number of boids in real time. Boids are added at random world-edge positions or removed randomly.
- Separation / Alignment / Cohesion weight (sliders, range 0–15): adjust the relative influence of each steering behavior immediately.

- **START:** begins or resumes the simulation loop.
- **PAUSE:** freezes the simulation without losing state.
- **RESET:** restores the flock to the state that was active when the application started (or when the last file was loaded).
- **QUIT:** closes the application; prompts if there are unsaved changes.

Menu Bar

- **File → Open...:** opens a native file chooser to load a JSON flock file.
- **File → Save:** saves the current flock to `data/save.json` after a confirmation prompt.
- **File → Save As...:** opens a native file chooser to save to any location.
- **View → HUD:** toggles the heads-up display overlay (boid count, FPS, run status).
- **View → Theme → Dark / Light:** switches the entire UI theme.

HUD Overlay

The top-left corner of the canvas shows a semi-transparent panel with three live metrics: BOIDS (current count), FPS (smoothed over 30 frames), and STATUS (RUNNING / PAUSED).

3. Program Structure

The codebase is split into two packages:

- `flock.logic`: all simulation logic, independent of the GUI.
- `flock.UI`: all ScalaFX user-interface classes.

Logic Layer (`flock.logic`)

Vector2D

An immutable case class representing a 2D vector. It provides arithmetic operators (`+`, `-`, `*`), `magnitude()`, `magnitudeSquared()`, `normalize()`, `truncate(maxLength)`, `dot(v)`, and `angle(v)`. Circe codecs are derived automatically in the companion object. This class is the mathematical backbone used by every other logic class.

Boid

A case class holding mutable `_position` and `_velocity`, plus internal `acceleration` and `steeringForce` accumulators. Key methods:

- `applyForce(f)` accumulates steering forces
- `update(dt, constants)` integrates position and velocity using the vehicle model and resets accumulators

- `wrapAround(constants)` applies toroidal boundary conditions
- `distanceSquared(other)` and `angle(other)` support neighbor queries.

Constants

An immutable case class holding all simulation parameters (perception radius, perception angle, speeds, weights, world dimensions, flock size limits). Defaults are provided in the companion object as `Constants.default`.

SteeringBehavior (trait)

A trait defining the interface for a steering behavior:

- `apply(boid, allBoids, c)` computes a raw force vector
- `weight(c)` returns its scalar multiplier from `Constants`.
- The protected helper `perceptionNeighbors(boid, allBoids, c)` filters neighbors by perception radius (using squared distance) and viewing angle.

Three singleton objects implement this trait:

- **Separation:** repels from overly close neighbors using inverse-squared distance weighting.
- **Alignment:** steers toward the average velocity of perceived neighbors.
- **Cohesion:** steers toward the average position (center of mass) of perceived neighbors.

Flock

Owns an `ArrayBuffer[Boid]` and a sequence of `SteeringBehavior` instances. `update(dt)` runs two passes: first accumulating all weighted steering forces for every boid (using the positions at the start of the tick), then integrating each boid's state.

Additional methods:

- `addBoid, removeBoid, addRandomBoid` (spawns at a random world edge)
- `addRandomBoids(n) / removeRandomBoids(n)` (capped by `min/maxFlockSize`)
- `setSize(n), resetWith(seq)`
- `updateConstants` (validates invariants before replacing `_constants`).

FlockFileIO

A singleton object with two methods that return `Try`:

- `loadFlockFromFile(path)` reads a JSON array of boids using Circe
- `saveFlockToFile(flock, path)` serializes the flock's boids to indented JSON.

UI Layer (flock.UI)

FlockWindow

A `Pane` subclass that renders boids as `Polygon` arrow shapes managed in a shape pool.

- `sync(boids)` adds or removes polygon nodes to match the boid count
- `render(boids)` updates each polygon's translation and rotation
- A clipping rectangle prevents drawing outside the canvas boundary
- `updateHud` and `toggleHud` control the HUD overlay.

ControlPanel

A `GridPane` with four `Setting` rows (one `SpinnerSetting` for flock size, three `SliderSetting` for weights) and a column of four action buttons. All interaction is exposed via callback-registration methods (`onStart`, `onPause`, etc.) so that `SimulationScene` can wire behavior without knowing widget internals.

FlockMenuBar

A `MenuBar` subclass with File and View menus. Theme selection uses a `ToggleGroup` to ensure mutual exclusivity. All items are exposed via callback-registration methods.

SimulationScene

The top-level `Scene`, acting as a facade over `FlockWindow`, `ControlPanel`, and `FlockMenuBar`. It handles theme swapping (stylesheet replacement) and file-dialog coordination, delegating all other calls to sub-components. `Main` only interacts with `SimulationScene`, keeping it decoupled from widget internals.

Main

The application entry point (`JFXApp3`). Loads the initial flock, constructs the scene, wires all callbacks, and starts an `AnimationTimer` that computes delta-time, calls `flock.update`, re-renders, and updates the HUD each frame.

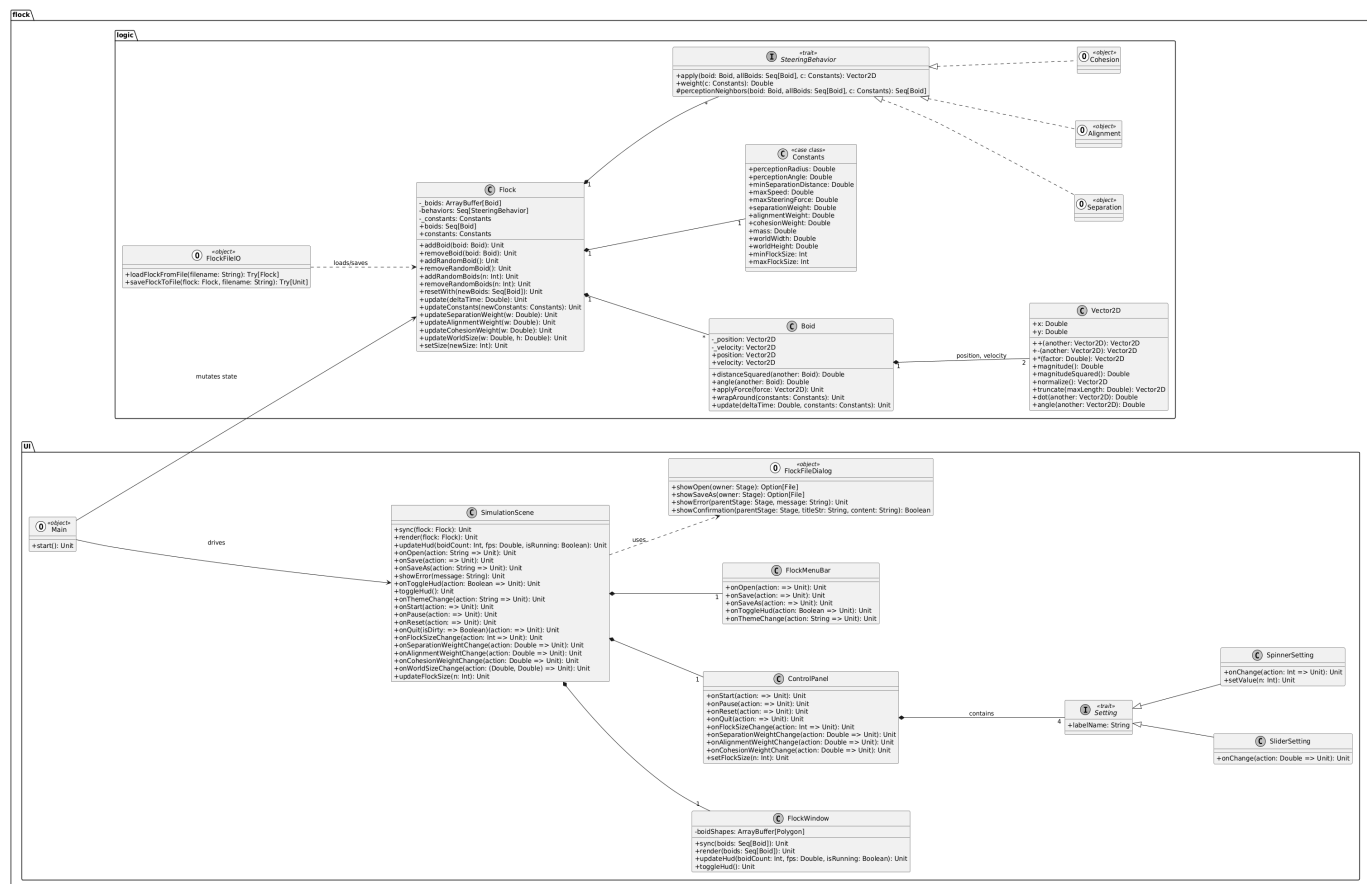
Setting / SliderSetting / SpinnerSetting

A trait `Setting` extends `HBox` to define a labeled control row. `SliderSetting` wraps a `Slider` with a value label and dynamic CSS gradient track fill. `SpinnerSetting` wraps an editable `Spinner[Int]`.

Class Diagram

(UML class diagram to be inserted here.)

4. Program structure



The code to generate this diagram is in `/data/diagram.puml`. You can go to <https://www.planttext.com> to generate one.

The application is divided into two packages to ensure a clean separation of concerns: `flock.logic` (computational core) and `flock.UI` (presentation layer).

Domain Model (flock.logic)

The logic layer models the physical and behavioral world of the simulation:

- **Vector2D:** An immutable case class that serves as the mathematical foundation for all position and force calculations
- **Boid:** Represents a single agent. It maintains state (position/velocity) and provides methods like `applyForce` and `update` to integrate movement
- **SteeringBehavior (Trait):** Defines the interface for flocking rules. Specific behaviors (Separation, Alignment, Cohesion) are implemented as singleton objects, making it easy to add new rules without modifying the `Flock` class

- Flock: The central manager of the Boid collection. Its key method, `update(dt)`, implements a two-pass update to ensure all boids compute forces based on the same world snapshot
- FlockFileIO: A utility object using the Circe library to handle JSON serialization and deserialization

User Interface (flock.UI)

The UI layer is built with ScalaFX and follows a modular component design:

- SimulationScene: Acts as a facade and coordinator between the menu bar, control panel, and the drawing canvas
- FlockWindow: A specialized pane that renders boids. It uses a shape pool (`ArrayBuffer[Polygon]`) to reuse JavaFX nodes, significantly improving performance by avoiding constant object allocation
- ControlPanel & FlockMenuBar: Encapsulate the interactive widgets (sliders, spinners, menus) and use callback registration to remain decoupled from the simulation logic
- Main: The entry point that initializes the app and drives the simulation using an `AnimationTimer`

Structural Justification

The most significant architectural choice was extracting steering logic into the `SteeringBehavior` trait hierarchy. While the original plan placed these as methods within `Flock`, this trait-based approach allows for superior testability and extensibility. Furthermore, using the `AnimationTimer` instead of a separate simulation thread eliminated complex synchronization issues (race conditions) since all state mutations occur on the JavaFX application thread.

5. Algorithms

Boids Flocking Model

The simulation implements the Boids model introduced by Reynolds [Reynolds 1987] with the autonomous steering formulation from [Reynolds 1999]. On each time step, every boid computes three independent steering forces from its locally perceived neighborhood.

Neighbor Detection

A boid b perceives another boid n as a neighbor if:

(1) $n \neq b$

(2) $\|pos(n) - pos(b)\|^2 \leq r^2$ (where r = perceptionRadius, compared using squared distance to avoid square roots)

(3) the angle between b 's velocity vector and the vector from b to n is at most $perceptionAngle / 2$. The current complexity is $O(N^2)$ per frame, which is acceptable for flock sizes up to approximately 500 boids.

Separation

For each neighbor n within `minSeparationDistance`, a repulsion vector is computed pointing from n toward the focal boid b and scaled by the inverse of the squared distance, so that nearer neighbors produce a stronger push:

$$repulsion(n) = (pos(b) - pos(n)) / \|pos(b) - pos(n)\|^2$$

The separation force is the sum of all such repulsion vectors. This continuous, distance-weighted formulation avoids abrupt transitions.

Alignment

The desired velocity is the mean velocity of all perceived neighbors. The steering force is the error between this desired velocity and the focal boid's current velocity:

$$F_{alignment} = (1/N) \sum vel(n_i) - vel(b)$$

Cohesion

The desired position is the center of mass of all perceived neighbors. The steering force points from the focal boid toward this center:

$$F_{cohesion} = (1/N) \sum pos(n_i) - pos(b)$$

Force Combination and Vehicle Model

The three forces are weighted by user-adjustable constants (w_s , w_a , w_c) and summed:

$$F_{total} = w_s \cdot F_{separation} + w_a \cdot F_{alignment} + w_c \cdot F_{cohesion}$$

The combined force is then truncated to `maxSteeringForce` before applying the forward Euler vehicle model:

$$F_{limited} = \text{truncate}(F_{total}, \text{maxSteeringForce})$$

$$acceleration = F_{limited} / mass$$

$$velocity_{new} = \text{truncate}(velocity + acceleration \cdot \Delta t, \text{maxSpeed})$$

$$position_{new} = position + velocity_{new} \cdot \Delta t$$

Δt is computed dynamically from the system clock (nanosecond precision), ensuring frame-rate-independent motion. If a boid's position exceeds the world boundary, modular wrap-around is applied on each axis independently.

Two-Pass Update

Flock.update executes two explicit passes per tick. In the first pass, forces are computed for all boids using the positions and velocities captured at the start of the tick, so that no boid's state affects the force computation of another within the same step. In the second pass, each boid integrates its accumulated force and updates its state. This prevents order-dependent artifacts that arise from single-pass naive implementations.

Pseudo-code: Two-Pass Flock Update

FOR each boid 'b' in the flock:

 Initialize total_force = (0, 0)

 neighbors = Find perceived neighbors of 'b' ($O(N^2)$)

 // Pass 1: Accumulate forces without changing state

 total_force += calculate_separation(b, neighbors) * separation_weight

 total_force += calculate_alignment(b, neighbors) * alignment_weight

 total_force += calculate_cohesion(b, neighbors) * cohesion_weight

 b.temporarily_force = truncate(total_force, max_force)

FOR each boid 'b' in the flock:

 // Pass 2: Integrate motion

 b.acceleration = b.temporarily_force / mass

 b.velocity = truncate(b.velocity + b.acceleration * delta_t, max_speed)

 b.position = b.position + b.velocity * delta_t

 apply_wrap_around(b.position)

6. Data Structures

ArrayBuffer[Boid] (in Flock)

The flock's boid collection is stored as a `scala.collection.mutable.ArrayBuffer[Boid]`. This structure provides $O(1)$ amortized append and $O(N)$ full traversal - both critical for the update loop that processes every boid every frame. A Scala `List` was considered but rejected because its $O(N)$ indexed access and poor cache locality would degrade performance during the force-accumulation pass. The mutable buffer also makes

dynamic flock resizing (add/remove individual boids) straightforward without allocating a new collection each frame.

Vector2D (immutable case class)

All positional, velocity, and force data are represented as immutable `Vector2D` instances. Immutability means that arithmetic methods always return new instances rather than mutating in place, which prevents aliasing bugs and makes reasoning about transformations easier. Boid positions and velocities are stored in mutable vars inside `Boid` itself, but each assignment replaces the reference atomically.

Constants (immutable case class)

All simulation parameters are packaged in an immutable `Constants` case class. When the user adjusts a slider or spinner, `updateConstants` creates a new `Constants` instance via `copy(...)` and replaces the reference in `Flock._constants`. Because the `AnimationTimer` runs on the JavaFX Application Thread and all UI callbacks also run on that thread, no additional synchronization is required - there is only one thread mutating state. This design is simpler and safer than the mutable map or synchronized block considered in the original plan.

Shape Pool (ArrayBuffer[Polygon] in FlockWindow)

Rendered boid shapes are managed as a pool of pre-allocated `Polygon` objects. The `sync` method adds or removes polygons only when the boid count changes; `render` only updates their `translateX`, `translateY`, and `rotate` properties each frame. This avoids the overhead of creating and garbage-collecting JavaFX nodes every tick.

FPS Rolling Buffer (Array[Double] of length 30)

Frame rate is smoothed over a fixed-size circular buffer of the 30 most recent raw FPS samples. Each frame, the new sample overwrites the oldest slot using a modular index, and the displayed FPS is the arithmetic mean. This eliminates display jitter from single-frame spikes without introducing latency.

7. Files and Internet Access

The application does not access the internet. All persistence is local JSON.

File Format

Flock state is stored as a UTF-8 JSON array. Each element is a JSON object with two nested objects, `"position"` and `"velocity"`, each containing `"x"` and `"y"` keys with double-precision floating-point values.

Example (two boids):

```
[{"position":{"x": 100.0,"y": 150.0},
  "velocity":{"x": 1.0  ,"y": -1.0}},
 {"position":{"x": 200.0,"y": 250.0},
  "velocity":{"x": -0.5 ,"y": 0.5  } }]
```

Serialization and deserialization are handled by Circe with automatically derived codecs for `Vector2D` and manually written product codecs for `Boid`. Only `position` and `velocity` are serialized; the transient fields `acceleration` and `steeringForce` are excluded.

The original technical plan specified a plain-text CSV format. The switch to JSON was made because Circe provides robust error reporting, eliminates manual parsing code, and produces human-readable output that is easy to inspect and hand-edit.

Default and Save Paths

- Default startup flock: `data/testui.json`
- Quick save target: `data/save.json`
- Save As / Open: any user-chosen path via the native file dialog.

8. Testing

Unit Tests (ScalaTest)

Unit tests are written with ScalaTest [ScalaTest] using the `AnyFlatSpec` style and `Matchers`. Test files are located in `src/test/scala/flock/logic/`.

Vector2DTest

Covers all `Vector2D` operations: `+`, `-`, `*`, `dot`, `magnitude`, `magnitudeSquared`, `normalize` (including zero-vector and negative-coordinate cases), `truncate` (limiting and non-limiting cases), and `angle` (including perpendicular vectors, same vector, and zero-vector inputs). Floating-point comparisons use a ± 0.0001 tolerance.

BoidTest

Covers squared-distance computation, view-angle computation (behind, ahead, and off-axis cases), `update` with zero delta-time (no change expected), max-speed truncating, force-then-update integration (manually traced arithmetic), steering-force reset between ticks, and huge-force truncating via `maxSteeringForce`.

FlockTest

Covers: `addBoid` / `removeBoid`; individual steering behavior outputs (separation direction, alignment error, cohesion direction); `update` position integration with zero steering force; velocity truncating to `maxSpeed`; edge case of two boids at the same position; `addRandomBoids` / `removeRandomBoids` respecting `min/maxFlockSize`; and `resetWith`.

FlockFileOTest

Covers: successful load from `src/test/resources/test1.json` (verifies boid count and first position); failure on nonexistent file; failure on a malformed JSON file (`src/test/resources/invalid.json`); and a round-trip save-then-load that verifies the saved flock's first boid position matches.

Manual / Visual Testing

The following scenarios were tested interactively:

- High Separation weight: boids maintain clear personal space with no overlap.
- Cohesion weight = 0: flock disperses, no grouping tendency.
- Rapid slider changes during live simulation: no crash, smooth behavioral transitions.
- Scaling to 200 boids: acceptable frame rate (≥ 30 FPS on development hardware). Currently, it still maintains a good frame rate (60 FPS).
- Boundary wrap-around: boids reaching a screen edge re-enter from the opposite side correctly.
- Load malformed JSON: graceful error dialog, simulation state unchanged.
- Theme switching: dark $\leftrightarrow$ light without visual artifacts.
- Save / Save As / Open: round-trip fidelity verified by visual inspection.

Test Coverage and Gaps

All unit tests pass.

The testing process deviated from the initial plan in its reliance on manual visual verification for the UI layer. While the plan originally aimed for broader coverage, the complexity of headless ScalaFX testing led me to prioritize rigorous unit testing for the core logic (Vector2D and Boid) instead. This trade-off ensured mathematical correctness while allowing for rapid UI iteration.

9. Known Bugs and Missing Features

Known Bugs

- None identified that reproduce reliably during testing.

Missing Features / Limitations

- No spatial partitioning (e.g., grid or quadtree): neighbor search is $O(N^2)$, which becomes noticeable above approximately 300 boids on modest hardware. Adding a grid hash would reduce this to approximately $O(N)$ per frame.
- No obstacle avoidance behavior: the simulation is purely open-world.
- No predator/prey dynamics or goal-seeking behaviors, which were never part of the plan but are natural extensions of the Reynolds model.
- The RESET button restores the state at launch (or last file load), not a user-defined checkpoint; there is no 'undo' history.

10. Three Best Sides and Three Weaknesses

Best Sides

- Clean separation of concerns: the logic package has zero dependencies on ScalaFX; it can be tested and reused independently. The SteeringBehavior trait makes adding a new rule (obstacle avoidance, goal-seeking) a matter of implementing one object without touching Flock or any existing class.

- Robust file I/O: using Circe gives structured error messages for malformed files, a clean codec definition with no manual parsing code, and a format that is human-readable and easy to create test fixtures for.
- Two-pass update correctness: separating force accumulation from state integration guarantees that all boids compute their forces against the same snapshot of the world, eliminating order-dependent artifacts that are a common and subtle bug in naïve flocking implementations.

Weaknesses

- $O(N^2)$ neighbor search: there is no spatial partitioning. Performance degrades visibly above ~300 boids, limiting the simulation's scalability.
- No automated GUI tests: the entire UI layer is covered only by manual testing. A headless ScalaFX test harness or a model - view split would address this.
- Perception angle is treated as symmetric around the velocity heading: biologically, a field of view often has a blind spot directly behind the agent. The current formulation uses the absolute angle, so a boid directly behind is excluded only if the perceptionAngle is less than 360° , which is correct, but the angle calculation uses the full cone without a distinct blind-spot region.

11. Deviations from Plan, Process, and Schedule

The implementation broadly followed the plan's two-week sprint structure.

- Weeks 1–2: Vector2D and Boid were implemented and tested ahead of schedule (~7 hours instead of the planned 20). The extra time was redirected to exploring Circe for JSON from the start.
- Weeks 3–4: Flock with steering behaviors as methods were implemented and validated through unit tests.
- Weeks 5–6: GUI construction took significantly more the planned time (~36 hours instead of the planned 18). The decision to use AnimationTimer instead of a dedicated simulation thread (planned) saved ~8 hours of threading complexity and eliminated all synchronization concerns.
- Weeks 7–8: The extraction into a SteeringBehavior trait hierarchy was a significant structural deviation from the plan (which placed these as Flock methods) and added roughly 4 hours but improved testability remarkably. Dark/light theme support and HUD were added beyond the original plan (~6 hours).

Overall, the total effort was close to the lower bound of the 115–130-hour estimate. The largest deviations from the plan were architectural: the threading model and the steering-behavior extraction.

The most valuable lesson was that designing for testability (separating logic from UI, using immutable data) paid dividends early: unit tests caught several off-by-one errors in the steering calculations before the GUI was ready for visual verification.

12. Final Evaluation

The final program is a complete, interactive, and visually polished flocking simulation. Core correctness goals are met: the three steering behaviors produce recognizable emergent flocking, parameter changes take effect immediately, and file I/O is reliable with graceful error handling.

The main technical weakness is scalability: the $O(N^2)$ neighbor search limits the useful flock size. Spatial partitioning is the obvious next step. A secondary weakness is the absence of automated UI tests; refactoring FlockWindow to separate rendering logic from JavaFX node management would make it more testable.

The choice to use an AnimationTimer rather than a separate simulation thread simplified the architecture considerably. However, it ties simulation speed to rendering frame rate; a decoupled design would allow the simulation to run at a fixed physics rate regardless of display refresh rate, which would be preferable for reproducibility.

If starting again, the two changes I would make are:

(1) adopt spatial partitioning from the beginning, since it is straightforward to add a grid hash alongside the ArrayBuffer

(2) design the rendering layer with a cleaner model - view separation to enable automated testing of the UI logic.

13. References and External Code

References

- Reynolds, C. W. (1987). Flocks, Herds, and Schools: A Distributed Behavioral Model. ACM SIGGRAPH Computer Graphics, 21(4), 25–34.
- Reynolds, C. W. (1999). Steering Behaviors For Autonomous Characters. <http://www.red3d.com/cwr/steer/>
- ScalaFX documentation. <https://www.scalafx.org/>
- Circe JSON library. <https://circe.github.io/circe/>
- ScalaTest. <https://www.scalatest.org/>

Libraries Used

- `io.circe / circe-core, circe-generic, circe-parser`: JSON serialization and deserialization of flock state.
- `org.scalafx / scalafx`: GUI framework wrapping JavaFX.
- `org.scalatest / scalatest`: unit testing framework.

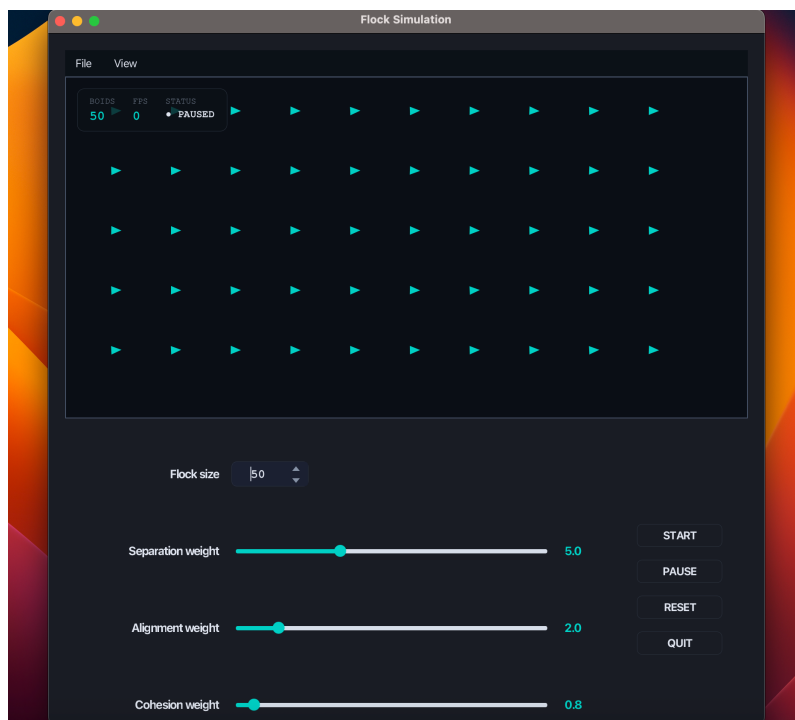
AI and External Code

Gemini was used for assistance in creating the CSS stylesheet for the two themes. I believe this is just the style preference problem, unrelated to the UI frame.

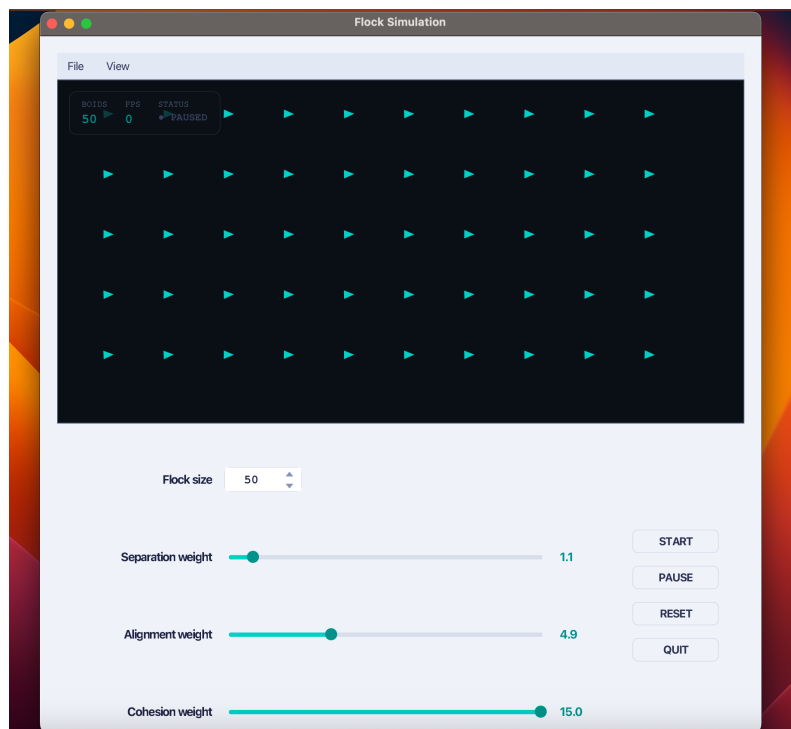
14. Appendices

Some screenshots of different situations when executing the program:

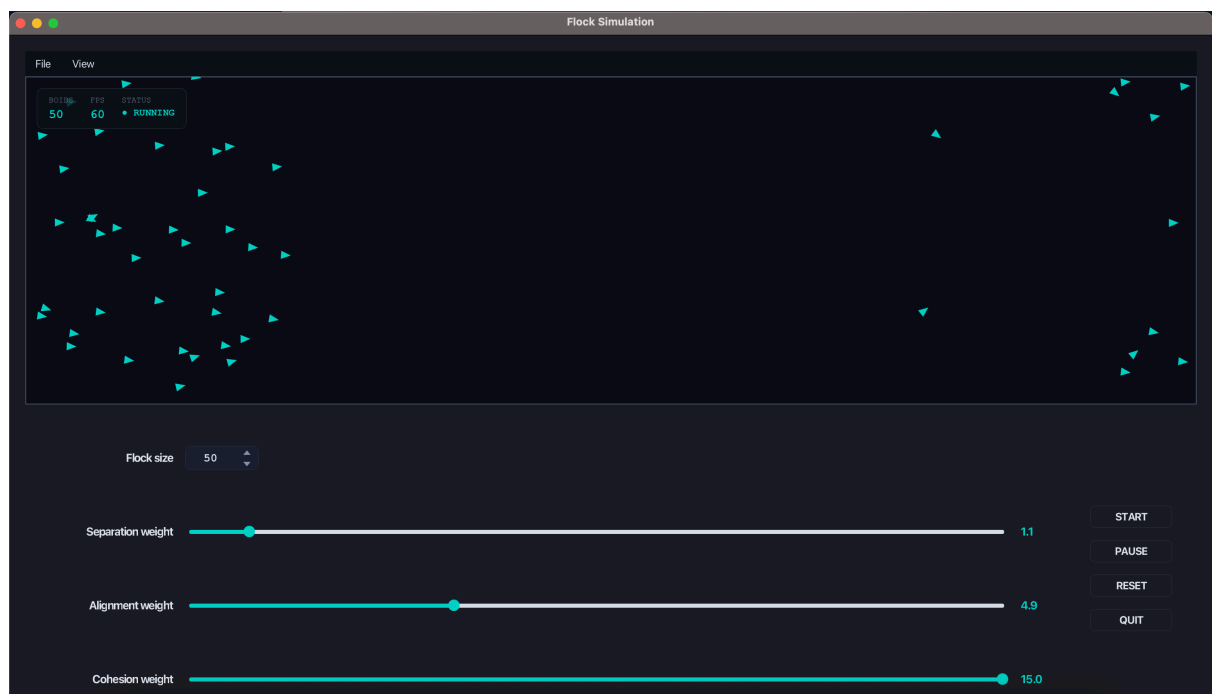
When the program runs:



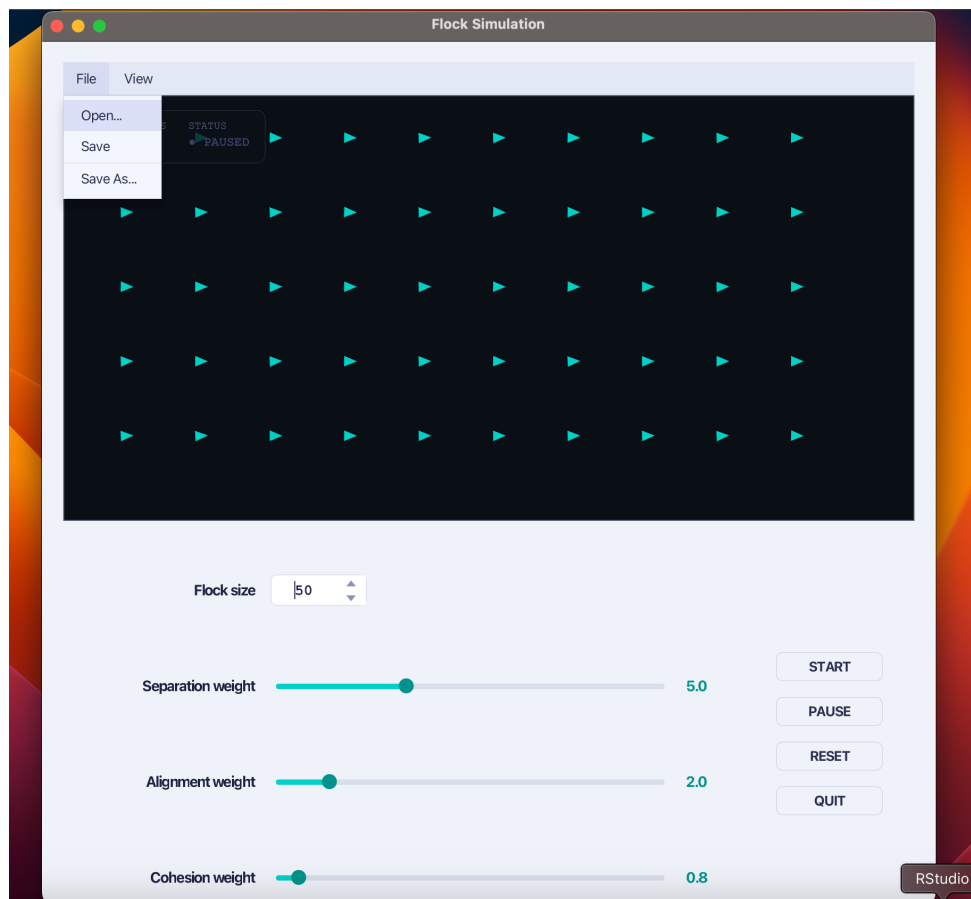
Light theme:



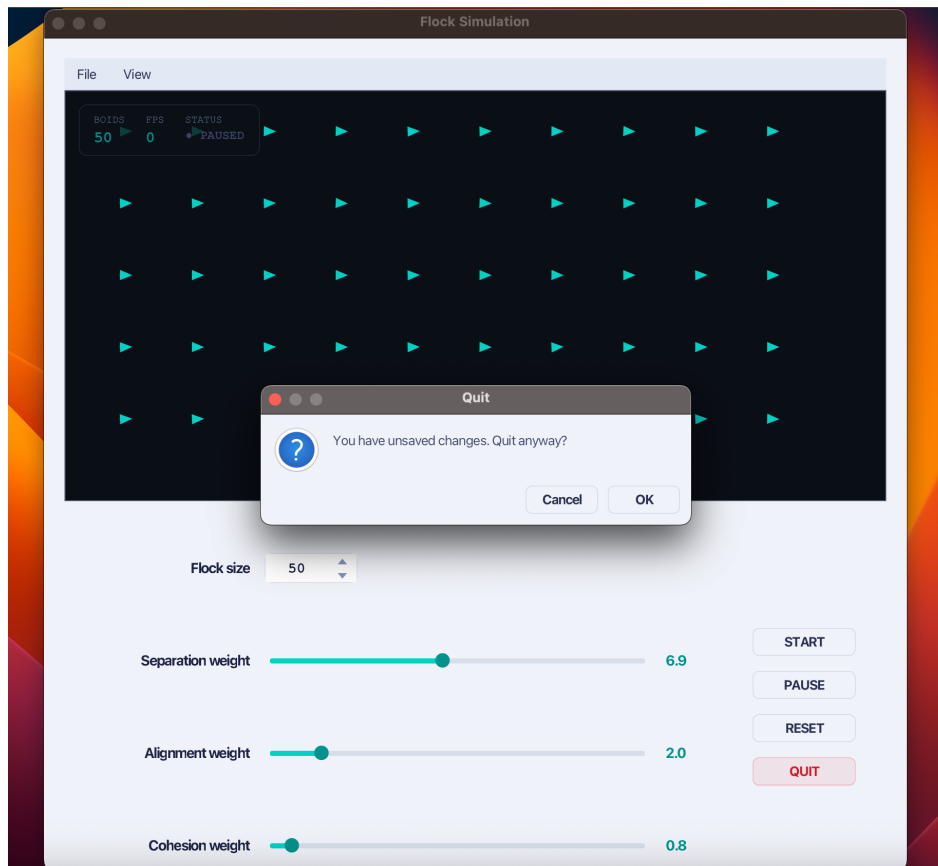
Expanded window:



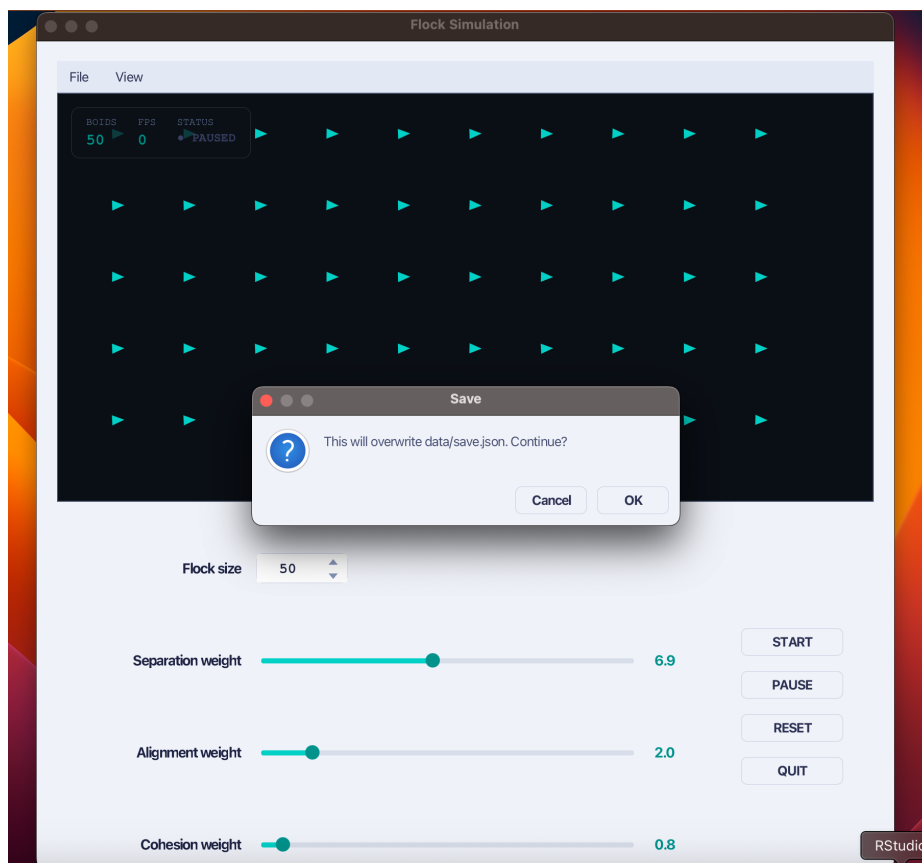
File Open/Save/Save As:



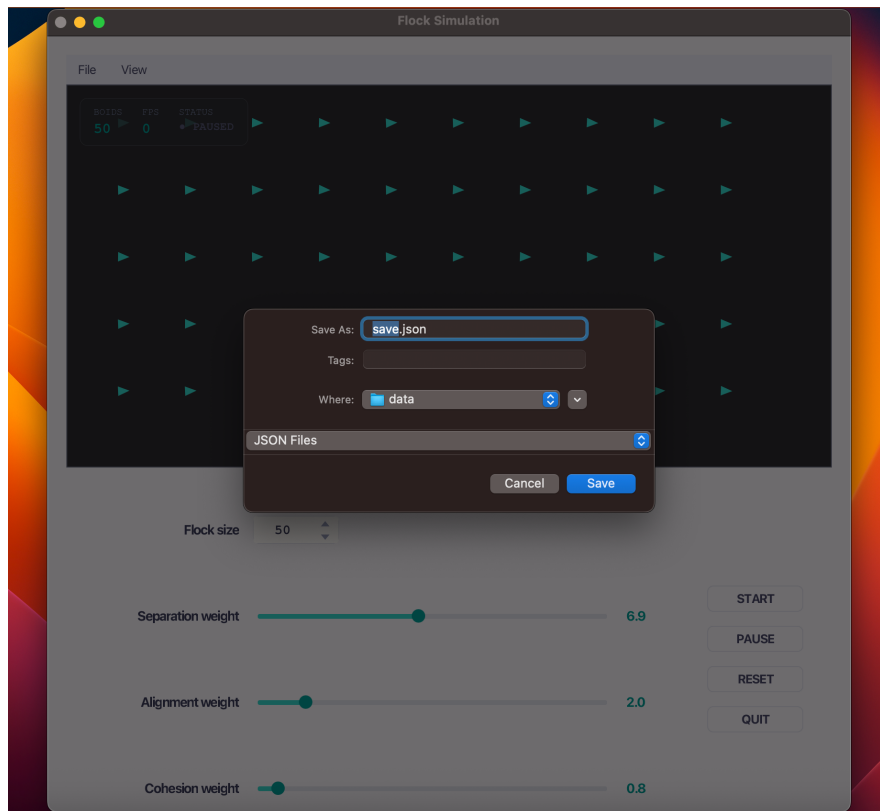
Quitting with edited weight/flock size will prompt a confirmation message:



Quick save (overwriting save.json) will also prompt a confirmation message:



Save as option allows setting file name (“save.json” as default) and path (data as default):



Opening a malformed file:

